CODENAMES

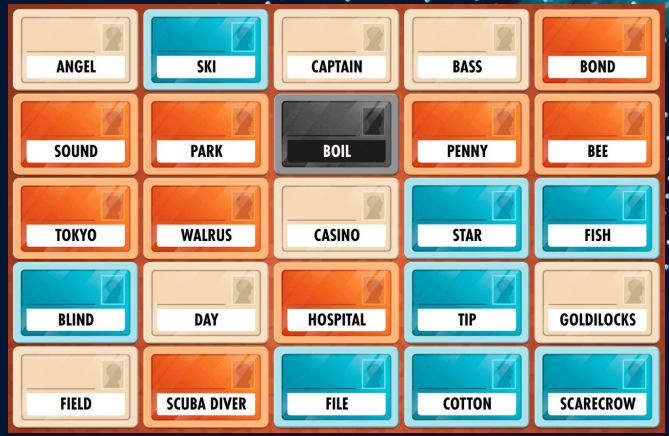
MDST

Our Goal

Make an Automated Spymaster for Codenames!

What is Codenames?

- **Objective:** Correctly guess all of your team's words
- 2 teams, **5x5 board**, random words
    - each team assigned **8-9 words**; One **assassin word**
        - Lose game automatically if assassin chosen!
- One Spymaster on each team to give clues
    - **One word clue** and associated **number**
- Players guess words based on the clue



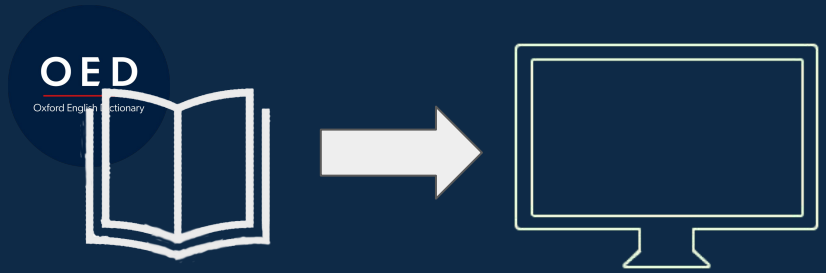
Dataset

IMPLEMENTED DATASETS

For The Learning Model

We took in the **English Dictionary** and performed data cleaning to get the **set of all English words**

- We **trained** our models on this dataset



From Codenames

We also got the datasets of **all possible Codenames words (~400)** and the dataset of **words in each Codenames game**

```
ANTARCTICA
APPLE
ARM
ATLANTIS
AUSTRALIA
AZTEC
BACK
BALL
```

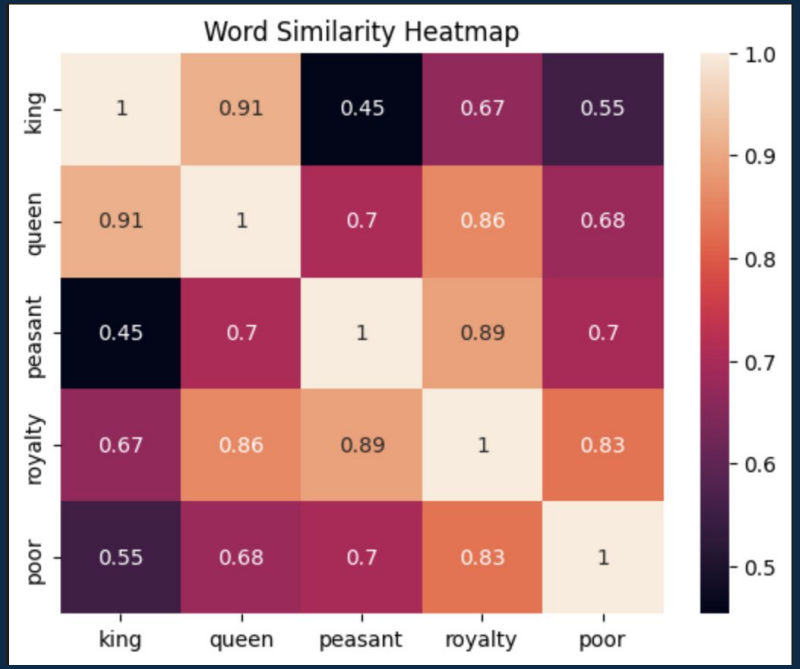
Data Cleaning

- Removed any non-letter characters, whitespace, commas, etc.
- Tokenized the definitions
- Lemmanize words to remove plurals

Methods

Word2Vec

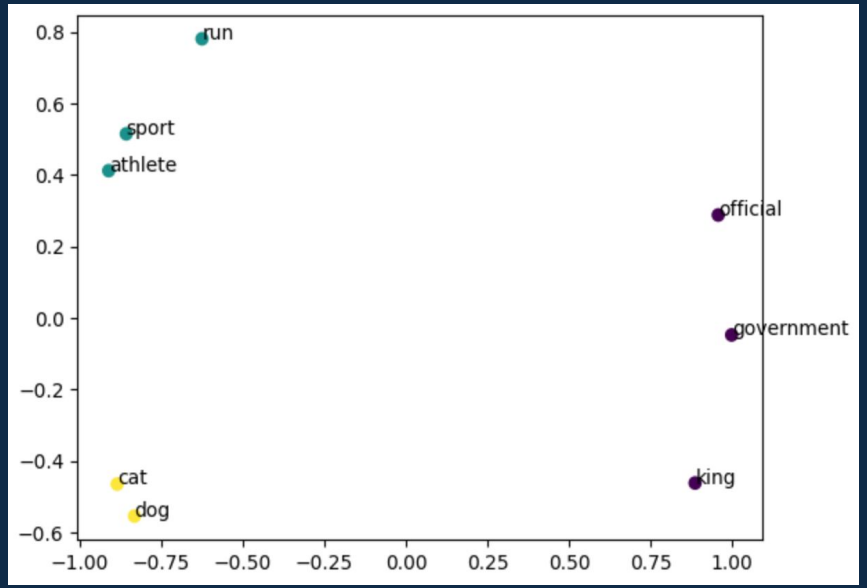
- Representing words as vectors
- Generate similarity scores between words



Heatmap to show similarity scores between words

K-Means Clustering

- Groups similar data into k different clusters, by similarity
- Find the average word vector for the cluster
- Find the most similar word to each cluster



Example of k-means clustered words

Q-learning

- Iteratively improve through reinforcement learning
- Rewards good guesses, penalizes bad ones
  - Penalized clues too similar to opponents'
- Enhance our initial Word2Vec model
  - Utilized a simplified Q-Learning approach

```python
def QLClueMaster(board, bluelist, redlist, assassin, model):
    #create the board master list with wordvectors and penalties
    board_max = []
    for word in board:
        if word in bluelist:
            penalty = 100
        elif word in redlist:
            penalty = -100
        elif word in assassin:
            penalty = -1000
        else:
            penalty = -50
    board_max.append([word, model.wv[word], penalty])
```

Example of Q-Learning being used

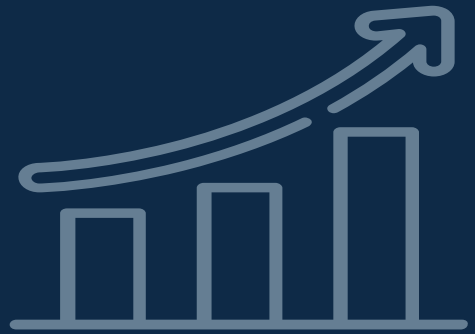
```
Clue: watch
Score: -15.625883142153421
Word List: ('time', 'sub', 'slug', 'port', 'ring', 'ambulance')
```

Example of Q-Learning output

Results

STRENGTHS

- Accounts for opposition's words and "assassin"
- Does well when giving clues for 1 word
- Model is able to give clues connecting up to 6 words

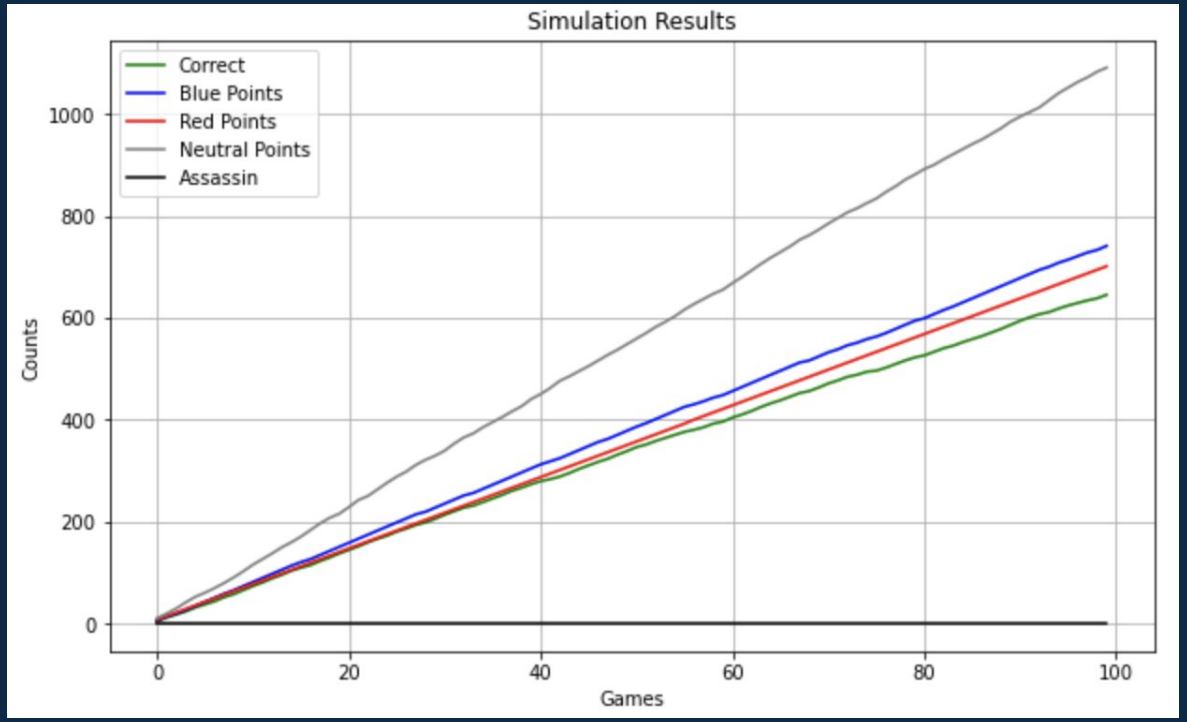


AREAS FOR IMPROVEMENT

- Typically loses to human-led opposition 😕
  - Played Codenames vs. the models to test
- Uncommon words given as clues
  - Ex: “gar” was given as a clue connecting the words “banana” and “shark”
- Some words not present in our dictionary (e.g. country names)

Testing Bot

- Created a tester bot simulate games
- We assumed Team Blue
- We typically score more points than the other team, but its marginal



Example of 100 simulated games using our “tester”

Future Goals & Reflection

CHALLENGES

- Only 7 weeks of work sessions
- The data we were training our models on
- Nature of Codenames allows for “thinking outside the box” – which is challenging for our models

Things We Could Investigate Further

- Expanding to a fully-functioning interactive with spymasters and guessers!
- Basic Q-learning worked well
- More complex models – e.g. GPT model, deep learning

Thank You!